Machine Learning and Data Processing

a.a. 2024-2025

Apprendimento non supervisionato

Prof. Ing. F. Benedetto

francesco.benedetto@uniroma3.it
fbenedetto@ieee.org

Chair of "Machine Learning and Data Processing" – University of Roma Tre
Chair of the "Definitions and Concepts for Dynamic Spectrum Access" - IEEE 1900.1

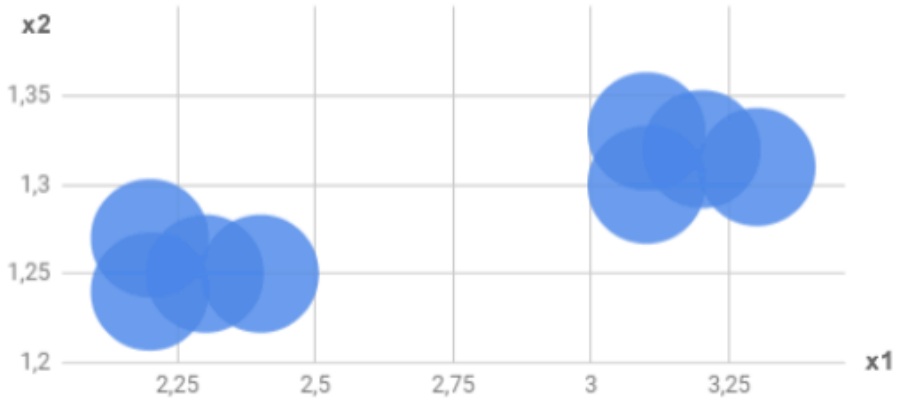
©SP4TE Lab 2017

Clustering

- Il clustering è una tecnica non-supervisionata di machine learning che di solito viene usata per effettuare analisi statistiche dei dati su vari campi e domini applicativi.
- L'obiettivo è quello di dividere l'insieme delle osservazioni in più sottoinsiemi, detti cluster o gruppi. Le osservazioni che appartengono allo stesso gruppo hanno in comune caratteristiche simili che differenziano anche i cluster tra di loro.
- Esempio: In questi dati sono presenti tre caratteristiche x1, x2, x3.

x1	x2	x3
2,4	1,25	1
3,1	1,3	2
3,3	1,31	2
2,2	1,24	1
2,3	1,25	1
3,2	1,32	2
2,2	1,27	1
3,1	1,33	2

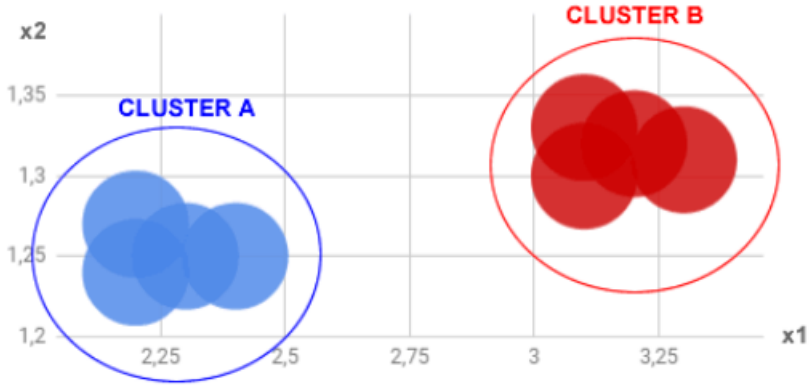
La macchina non ha informazioni sulle etichette e nessuna funzione di apprendimento. Non c'è supervisione.
Ciò nonostante, nella tabella c'è una **relazione significativa tra x_1, x_2, x_3**.
Per vederla meglio, escludiamo momentaneamente il dato x_3 e rappresentiamo i dati x_1, x_2 su un diagramma cartesiano. Nel grafico a due dimensioni comincia già a vedersi una regolarità e uno **schema nei dati**.



Clustering

- Poi associamo un colore diverso (blu, rosso) alle coordinate (x1, x2) per indicare il terzo dato x3 ossia la terza dimensione.
- Il blu per il valore x3=1 e il rosso per il valore x3=2.

x1	x2	x3
2,4	1,25	1
3,1	1,3	2
3,3	1,31	2
2,2	1,24	1
2,3	1,25	1
3,2	1,32	2
2,2	1,27	1
3,1	1,33	2



Ora il raggruppamento (cluster) è subito evidente anche a occhio nudo.

Nel cluster A e nel cluster B sono raggruppati dei dati simili.

In questo modo la macchina ha appreso un'informazione rilevante dai dati, senza alcun aiuto da parte del supervisore.

Nota: Questo è un semplice esempio a due dimensioni ma rende l'idea del funzionamento. In realtà, il clustering è particolarmente utile quando è applicato a banche dati multidimensionali. Dove l'occhio umano non può fare nulla.

Nel machine learning gli algoritmi di clustering sono usati per individuare le relazione tra i dati tramite un processo matematico-statistico di apprendimenti.

K-Means

- Tra le diverse tecniche di apprendimento non supervisionato, l'algoritmo K-Means è uno dei più utilizzati e apprezzati per il clustering dei dati.

- K-means: un algoritmo di clustering partizionale che ha dimostrato di essere efficace nella suddivisione dei dati in gruppi omogenei.

- Partizionale significa che, a ogni passo, lavora su una porzione diversa di dati. Questo lo differenzia dai metodi gerarchici che invece trovano nuovi cluster basandosi su cluster precedentemente stabiliti.

- K-means si basa sui cosiddetti centroidi.

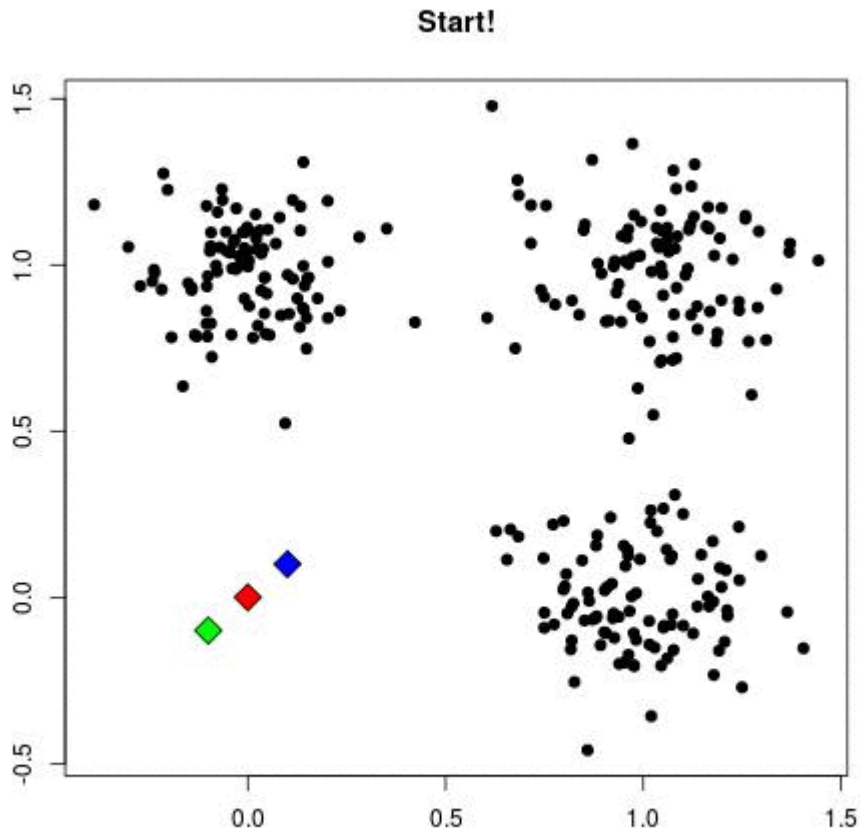
- Il centroide è un punto appartenente allo spazio delle features che media le distanze tra tutti i dati appartenenti al cluster ad esso associato. Rappresenta quindi una sorta di baricentro del cluster ed in generale, proprio per le sue caratteristiche, non è uno dei punti del dataset.

- L'obiettivo di K-means è assegnare ogni punto dati al cluster più vicino rispetto al centroide di quel cluster, al fine di minimizzare l'errore quadratico totale

K-Means

Il processo di raggruppamento non supervisionato può essere descritto in cinque step:

1. Non conoscendo le classi presenti nel dataset di ingresso, la prima cosa da fare è decidere il numero di classi (o meglio cluster, in questo caso) in cui si vuole suddividere il dataset stesso. Questo numero è detto K, da cui il nome del metodo K-means (il termine means sottintende l'uso dei centroidi, cioè di punti medi).
2. Si scelgono in modo casuale K centroidi appartenenti allo spazio delle features. L'unica condizione è che non siano coincidenti, anzi solitamente ci si assicura che siano abbastanza distanti tra loro. In caso contrario l'algoritmo potrebbe avere problemi a convergere.
3. Si calcola la distanza di ogni punto del dataset rispetto ad ogni centroide.
4. Ogni punto del dataset viene associato al cluster collegato al centroide più vicino.
5. Si ricalcola la posizione di ogni centroide facendo la media delle posizioni di tutti i punti del cluster associato (solo di questi punti!)
6. Si itera dal punto 3 fino a quando non ci sarà più alcun ingresso che cambia di cluster.

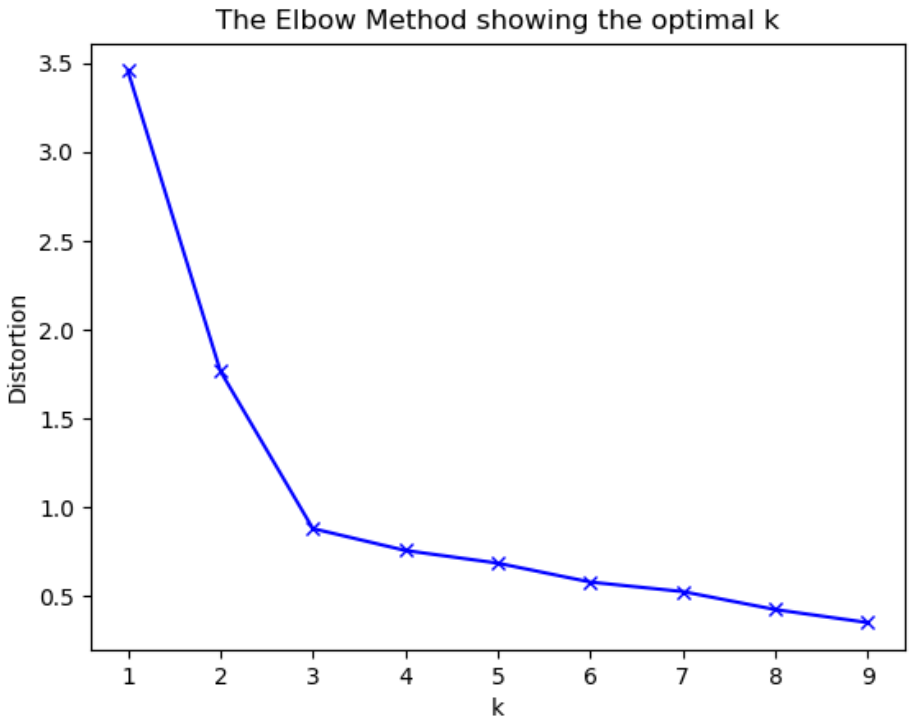


K-Means: iper-parametro K

I dati di ingresso dell'animazione precedente sono tali da costituire 3 raggruppamenti già individuabili ad occhio. In generale la situazione non è così semplice e quindi potrebbe non essere così immediato individuare il numero di cluster da utilizzare.

E' però possibile utilizzare un metodo maggiormente oggettivo per decidere questo numero: il cosiddetto elbow method o metodo del gomito.

In pratica si itera il K-means per diversi valori di K ed ogni volta si calcola la somma delle distanze al quadrato tra ogni centroide ed i punti del proprio cluster.



Graficando i valori di K (asse orizzontale) e i valori della somma delle distanze al quadrato (asse verticale), si ottiene un grafico simile a quello in figura. Questo grafico deve essere letto da destra verso sinistra. Si deve trovare il punto in cui la curva tende a salire in modo più consistente.

Nella figura in esempio si nota che dal valore K=9 a K=3 la curva sale in modo quasi lineare. Dal valore K=3 a K=1 la curva sale in modo più evidente. Ciò significa che il valore K=3 è il nostro gomito (pensando alla curva come ad un braccio con la mano in K=9, il gomito in K=3 e la spalla in K=1), da cui il nome del metodo.

Il numero ottimale di cluster è quello in cui è posizionato il gomito.

DBSCAN

DBSCAN è l'acronimo inglese di Density-based spatial clustering of applications with noise, un sofisticato algoritmo di clustering comunemente usato per il Data Mining e Machine Learning.

Uno dei meravigliosi vantaggi del **DBSCAN** è la capacità d'individuare cluster dalle forme arbitrarie senza rimanere influenzato dal rumore di fondo (*noise*), in quanto lavora sulla densità dei punti e non sulla loro distanza (permette la creazione anche di cluster non sferici).

Il funzionamento del **DBSCAN** si basa sul presupposto che se una particolare osservazione appartenesse a uno specifico *cluster*, dovrebbe essere vicina alla maggior parte dei **samples** appartenenti al medesimo cluster.

Nel caso del DBSCAN il numero di cluster non è necessario fornirlo in quanto sarà l'algoritmo stesso a scoprire quanti cluster esistono in base a due iper-parametri che verranno forniti:

- epsilon o eps (ε): la distanza minima che devono avere due dati in ingresso per essere considerati "vicini"
- minSamples: numero minimo di dati in ingresso, "vicini" tra loro, richiesto per formare un cluster. In generale a questo iper-parametro va dato un valore >= numero dimensioni dataset + 1. Ad esempio, se abbiamo un dataset costituito da valori in due dimensioni (punti su un piano), allora minSamples >= 3.

DBSCAN

Il DBSCAN itera sui punti del dataset in ingresso ed esegue delle valutazioni basate sui due iperparametri eps e minSamples. L'algoritmo si può riassumere all'incirca come segue:

```
per ogni punto p
n = get_numero_punti_vicini(p, eps)
se n >= minSamples allora
   p è core_point
   crea_cluster(p)
altrimenti
   cp = cerca_core_point_vicino(p, eps)
   se esiste(cp) allora
      p è border_point
      p assegnato_a_cluster(cp)
   altrimenti
      p è noise_point
```

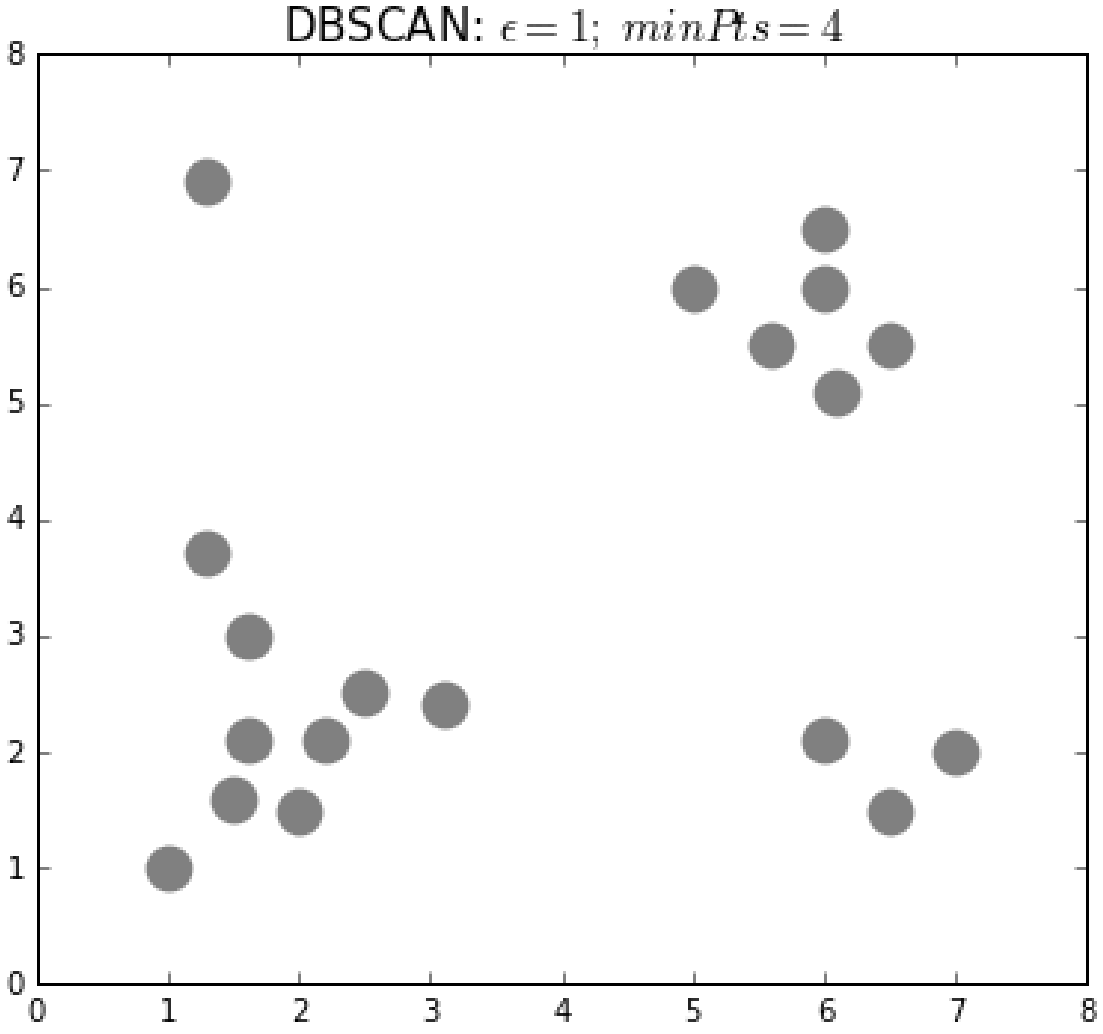
- Un **core point** è un punto che ha attorno a lui (entro la distanza eps) un numero di altri punti almeno pari a minSamples. Un core point definisce un cluster.

- Un **border point** è un punto attorno al quale si trovano un numero di punti minore di minSamples, ma uno di questi è un core point. Per questo motivo questo border point viene assegnato al cluster identificato dal core point a lui vicino.

- Un **noise point** è un punto attorno al quale si trovano un numero di punti minore di minSamples e NESSUNO di questi è un core point. Questo significa che il noise point è a distanza maggiore di eps da qualunque core point. Per questo motivo non viene associato a nessun cluster e resta per conto suo, come outlier, cioè come valore anomalo.

DBSCAN

```
per ogni punto p
n = get_numero_punti_vicini(p, eps)
se n >= minSamples allora
    p è core_point
    crea_cluster(p)
altrimenti
    cp = cerca_core_point_vicino(p, eps)
    se esiste(cp) allora
        p è border_point
        p assegnato_a_cluster(cp)
    altrimenti
        p è noise_point
```



PCA

PCA è un algoritmo di Machine Learning non supervisionato che tenta di ridurre la dimensionalità (numero di caratteristiche) in un set di dati pur mantenendo la maggior quantità possibile di informazioni.

Ciò è possibile individuando un nuovo insieme di caratteristiche denominate componenti, che sono composite delle caratteristiche originali non correlate le une alle altre. Inoltre, le componenti sono vincolate in modo che la prima di esse rappresenti la maggiore variabilità possibile nei dati, la seconda componente la seconda maggiore variabilità e così via.

Le 5 fasi per l'analisi dei componenti principali (PCA):

1. Standardizzare l'intervallo di variabili iniziali continue

2. Calcolare la matrice di covarianza per identificare le correlazioni

3. Calcolare gli autovettori e gli autovalori della matrice di covarianza

4. Creare un vettore di funzionalità per decidere quali componenti principali mantenere

5. Riformulare i dati lungo gli assi dei componenti principali

PCA

Le 5 fasi per l'analisi dei componenti principali (PCA):

1. Standardizzare l'intervallo di variabili iniziali continue

$$z = \frac{value - mean}{standard\ deviation}$$

2. Calcola la matrice di covarianza per identificare le correlazioni

$$\begin{bmatrix} Cov(x,x) & Cov(x,y) & Cov(x,z) \\ Cov(y,x) & Cov(y,y) & Cov(y,z) \\ Cov(z,x) & Cov(z,y) & Cov(z,z) \end{bmatrix}$$

Poiché la covarianza di una variabile con se stessa è la sua varianza (Cov(a,a)=Var(a)), nella diagonale principale (in alto da sinistra in basso a destra) abbiamo effettivamente le varianze di ciascuna variabile iniziale. E poiché la covarianza è commutativa (Cov(a,b)=Cov(b,a)), le voci della matrice di covarianza sono simmetriche rispetto alla diagonale principale, il che significa che le porzioni triangolari superiore e inferiore sono uguali.

In realtà è il segno della covarianza che conta:

- se positivo allora : le due variabili aumentano o diminuiscono insieme (correlate)
- se negativo allora: uno aumenta quando l'altro diminuisce (correlato inversamente)

La matrice di covarianza non è altro che una tabella che riassume le correlazioni tra tutte le possibili coppie di variabili.

PCA

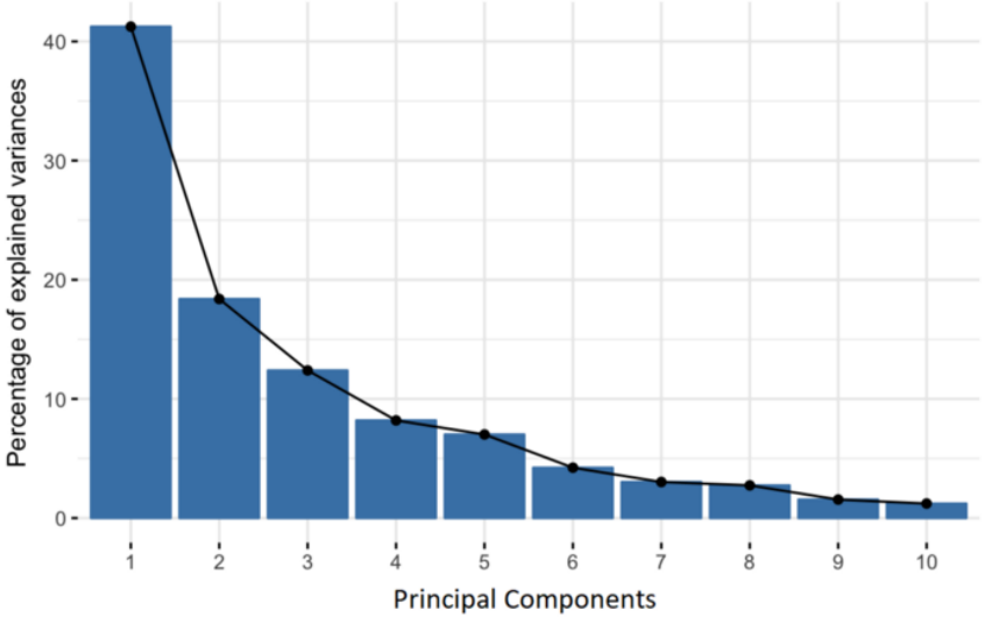
Le 5 fasi per l'analisi dei componenti principali (PCA):

3. Calcola gli autovettori e gli autovalori della matrice di covarianza: Autovettori e autovalori sono i concetti di algebra lineare che dobbiamo calcolare dalla matrice di covarianza per determinare le componenti principali dei dati.

Le componenti principali rappresentano le direzioni dei dati che spiegano una quantità massima di varianza, vale a dire le linee che catturano la maggior parte delle informazioni dei dati. La relazione tra varianza e informazioni qui è che, maggiore è la varianza portata da una linea, maggiore è la dispersione dei punti dati lungo di essa e maggiore è la dispersione lungo una linea, maggiore è l'informazione che ha. Per dirla semplicemente, basti pensare ai componenti principali come a nuovi assi che forniscono l'angolo migliore per vedere e valutare i dati, in modo che le differenze tra le osservazioni siano meglio visibili.

Gli autovettori della matrice di covarianza sono le direzioni degli assi dove c'è la maggior varianza (maggior parte delle informazioni) e che chiamiamo Componenti principali.

Gli autovalori sono semplicemente i coefficienti associati agli autovettori, che danno la quantità di varianza trasportata in ogni componente principale .



Le 5 fasi per l'analisi dei componenti principali (PCA):

4. Crea un vettore di funzionalità per decidere quali componenti principali mantenere.

In questo passaggio, quello che facciamo è scegliere se mantenere tutte le componenti principali o scartare quelle di minor significato (di bassi autovalori), e formare con le restanti una matrice di vettori che chiamiamo *Feature vector*.

Quindi, il vettore delle caratteristiche è semplicemente una matrice che ha come colonne gli autovettori delle componenti che decidiamo di mantenere. Questo lo rende il primo passo verso la riduzione della dimensionalità, perché se scegliamo di mantenere solo **p** autovettori (componenti) da **n** , il set di dati finale avrà solo **p** < **n** dimensioni.

5. Riformulare i dati lungo gli assi dei componenti principali.

Nei passaggi precedenti, a parte la standardizzazione, non si apportano modifiche ai dati, si selezionano solo i componenti principali e si forma il vettore delle caratteristiche, ma il set di dati di input rimane sempre in termini di assi originali (ovvero in termini di variabili iniziali).

In questo passaggio, che è l'ultimo, lo scopo è utilizzare il vettore di caratteristiche formato utilizzando gli autovettori della matrice di covarianza, per riorientare i dati dagli assi originali a quelli rappresentati dalle componenti principali (da cui il nome Analisi delle Componenti Principali). Questo può essere fatto moltiplicando la trasposizione del set di dati originale per la trasposizione del vettore della caratteristica.

$$FinalDataSet = FeatureVector^T * StandardizedOriginalDataSet^T$$

Confronti

	Apprendimento supervisionato	Apprendimento senza supervisione
In cosa consiste?	Si addestra il modello con un set di dati di input e un set corrispondente di dati di output etichettati accoppiati.	Si addestra il modello per scoprire modelli nascosti in dati non etichettati.
Tecniche	Regressione logistica, regressione lineare, albero decisionale e rete neurale.	Raggruppamento in cluster, apprendimento delle regole di associazione, densità di probabilità e riduzione della dimensionalità.
Obiettivo	Prevedere un output in base a input noti.	Identificare informazioni preziose sulle relazioni tra i punti dati di input. Questo può quindi essere applicato a nuovi input per ricavare informazioni simili.
Approccio	Ridurre al minimo l'errore tra gli output previsti e le etichette reali.	Trovare modelli, analogie o anomalie all'interno dei dati.

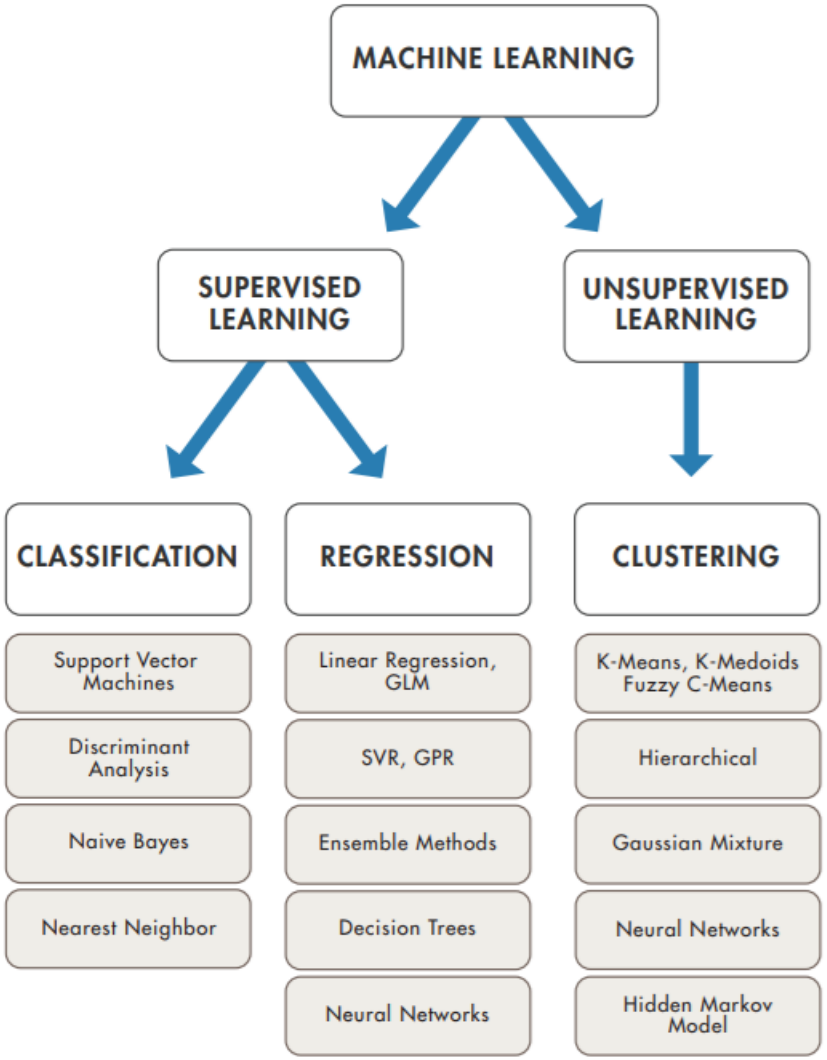
Come scegliere il modello

La scelta dell'algoritmo giusto può sembrare un'impresa ardua.
Sono dozzine gli algoritmi di machine learning supervisionati e non supervisionati, ed ognuno adotta un approccio diverso nella fase di apprendimento.

Non esiste un metodo migliore: esiste un “modello più adeguato” ai dati ed al contesto in cui andrà ad operare.

Choose supervised learning if you need to train a model to make a prediction--for example, the future value of a continuous variable, such as temperature or a stock price, or a classification—for example, identify makes of cars from webcam video footage.

Choose unsupervised learning if you need to explore your data and want to train a model to find a good internal representation, such as splitting data up into clusters.



Compromesso bias-varianza

bias basso, varianza bassa

bias alto, varianza bassa

bias basso, varianza alta

bias alto, varianza alta

Esso è la proprietà di un modello secondo cui la varianza del parametro stimato tra i campioni può essere ridotta aumentando il bias nei parametri stimati.

- L'errore di bias è un errore derivante da presupposti errati nell'algoritmo di apprendimento. Un elevato bias può far sì che un algoritmo manchi le relazioni rilevanti tra le caratteristiche e gli output di destinazione (underfitting).

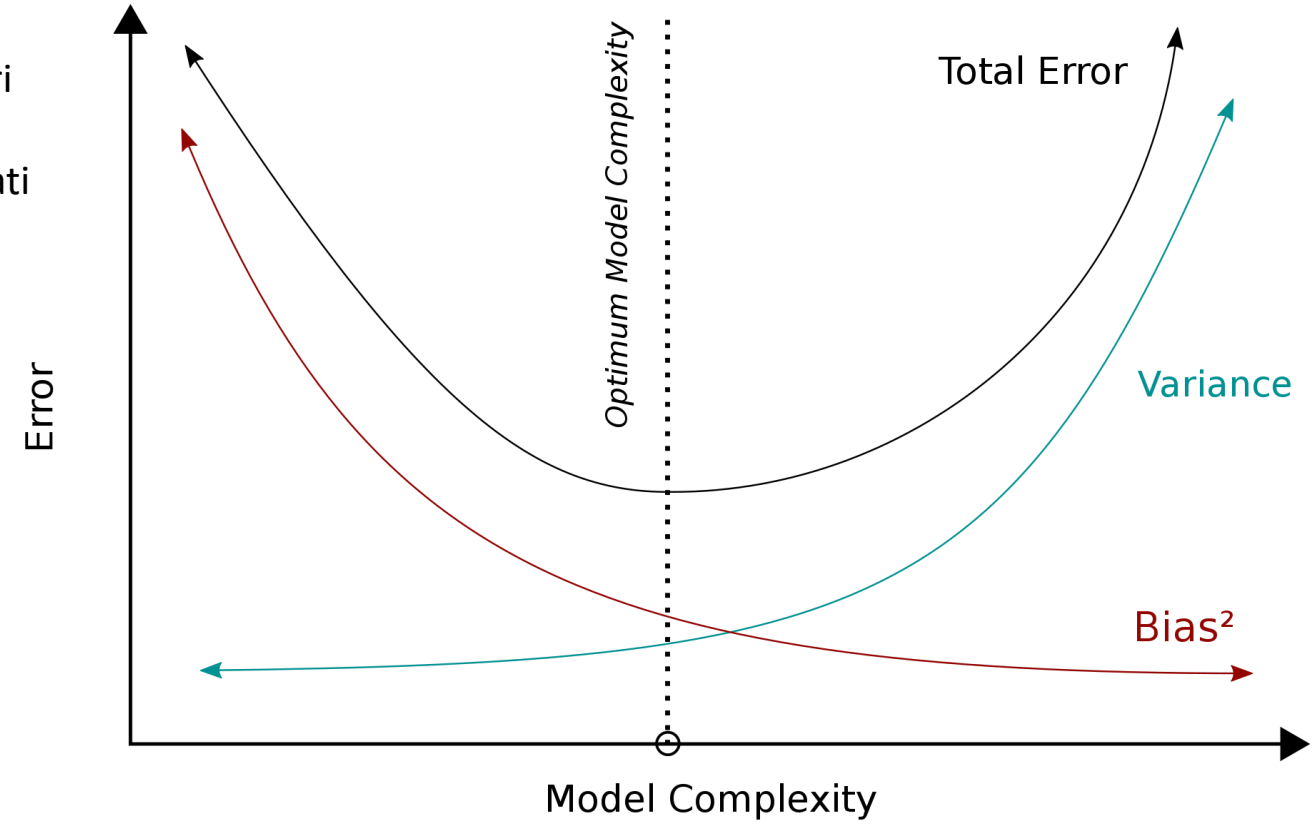
- La varianza è un errore dovuto alla sensibilità a piccole fluttuazioni nel training set. Un'elevata varianza può derivare da un algoritmo che modella il rumore casuale nei dati di addestramento (overfitting).

La scomposizione bias-varianza è un modo per analizzare l'errore di generalizzazione atteso di un algoritmo di apprendimento rispetto a un particolare problema come somma di tre termini, il bias, la varianza e una quantità chiamata errore irriducibile, risultante dal rumore nel problema stesso.

Compromesso bias-varianza

Il compromesso bias-varianza è un problema centrale nell'apprendimento supervisionato:

- Idealmente, si vuole scegliere un modello che catturi con precisione le regolarità nei suoi dati di addestramento, ma che generalizzi anche bene ai dati non visti.
- Sfortunatamente, in genere è impossibile fare entrambe le cose contemporaneamente.
- I metodi di apprendimento ad alta varianza possono essere in grado di rappresentare bene il loro set di training, ma rischiano di adattarsi eccessivamente ai dati rumorosi o non rappresentativi: OVERFITTING.
- Al contrario, gli algoritmi con alto bias producono in genere modelli più semplici che potrebbero non riuscire a catturare regolarità importanti (vale a dire underfit) nei dati: UNDERFITTING.



Compromesso bias-varianza

Si supponga di avere un training set $D = \{(x_1, y_1) \ldots, (x_n, y_n)\}$, composto da un insieme di punti $x_1, \ldots, x_n$ e valori reali y_i associato a ciascun punto x_i, e si assuma l'esistenza di una funzione $y = f(x) + \varepsilon$, dove il rumore ε ha media nulla e varianza σ^2.

L'obiettivo è trovare una funzione $\hat{f}(x; D)$ che approssima la vera funzione $f(x)$ nel miglior modo possibile, mediante un algoritmo di apprendimento basato sul training set. Nello specifico, "nel miglior modo possibile" significa richiedere che l'errore quadratico medio tra y e $\hat{f}(x; D)$, ovvero $(y - \hat{f}(x; D))^2$, sia minimo, sia per $x_1, \ldots, x_n$ *sia* per i punti al di fuori del campione. Naturalmente, dal momento che i valori y_i contengono rumore ε, ogni funzione che approssima avrà un "errore irriducibile".

Trovare una $\hat{f}$ che generalizza a punti al di fuori del set di addestramento può essere fatto con uno qualsiasi degli innumerevoli algoritmi utilizzati per l'apprendimento supervisionato. Ad ogni modo, per ogni funzione $\hat{f}$, è possibile il suo errore atteso su un nuovo campione x come segue:

$$\mathrm{E}_{D,\varepsilon}\left[\left(y - \hat{f}(x; D)\right)^2\right] = \left(\mathrm{Bias}_D\left[\hat{f}(x; D)\right]\right)^2 + \mathrm{Var}_D\left[\hat{f}(x; D)\right] + \sigma^2$$

dove

$$\mathrm{Bias}_D\left[\hat{f}(x; D)\right] = \mathrm{E}_D\left[\hat{f}(x; D)\right] - f(x)$$

e

$$\mathrm{Var}_D\left[\hat{f}(x; D)\right] = \mathrm{E}_D[\left(\mathrm{E}_D[\hat{f}(x; D)] - \hat{f}(x; D)\right)^2].$$

Il valore di espettazione spazia su diverse scelte del set di allenamento $D = \{(x_1, y_1) \ldots, (x_n, y_n)\}$, tutti campionati dalla stessa distribuzione congiunta $P(x, y)$. I tre termini rappresentano:

- il quadrato del *bias* del metodo di apprendimento, che può essere pensato come l'errore causato dalle ipotesi semplificative incorporate nel metodo. Ad esempio, quando si approssima una funzione non lineare $f(x)$ utilizzando un metodo di apprendimento per modelli lineari, ci saranno errori nelle stime $\hat{f}(x)$ a causa di questa ipotesi;
- la *varianza* del metodo di apprendimento, o, intuitivamente, quanto il metodo di apprendimento $\hat{f}(x)$ si muoverà intorno alla sua media;
- l'errore irriducibile σ^2 .

Poiché tutti e tre i termini non sono negativi, l'errore irriducibile forma un limite inferiore all'errore atteso su campioni invisibili.

Più complesso è il modello $\hat{f}(x)$ vale a dire, più punti dati acquisirà e minore sarà la distorsione. Tuttavia, la complessità farà "spostare" maggiormente il modello per acquisire i punti dati, e quindi la sua varianza sarà maggiore.

Compromesso bias-varianza: derivazione

La derivazione della scomposizione bias-varianza per l'errore al quadrato procede come segue. Per comodità di notazione, si abbrevia $f = f(x)$, $\hat{f} = \hat{f}(x; D)$ e si lascia cadere il pedice D sui nostri operatori di aspettativa. Innanzitutto, per definizione, per qualsiasi variabile casuale X, si ha

$$\mathrm{Var}[X] = \mathrm{E}[X^2] - \mathrm{E}[X]^2.$$

Riordinando, si ottiene:

$$\mathrm{E}[X^2] = \mathrm{Var}[X] + \mathrm{E}[X]^2.$$

Da quando f è deterministico, cioè indipendente da D ,

$$\mathrm{E}[f] = f.$$

Così, dato $y = f + \varepsilon$ e $\mathrm{E}[\varepsilon] = 0$ (perché ε è rumore), implica $\mathrm{E}[y] = \mathrm{E}[f + \varepsilon] = \mathrm{E}[f] = f$.

Inoltre, poiché $\mathrm{Var}[\varepsilon] = \sigma^2$,

$$\mathrm{Var}[y] = \mathrm{E}[(y - \mathrm{E}[y])^2] = \mathrm{E}[(y - f)^2] = \mathrm{E}[(f + \varepsilon - f)^2] = \mathrm{E}[\varepsilon^2] = \mathrm{Var}[\varepsilon] + \mathrm{E}[\varepsilon]^2 = \sigma^2 + 0^2 = \sigma^2.$$

In definitiva, poiché ε e $\hat{f}$ sono indipendenti,

$$\begin{aligned}
\mathrm{E}\left[(y - \hat{f})^2\right] &= \mathrm{E}\left[(f + \varepsilon - \hat{f})^2\right] \\
&= \mathrm{E}\left[(f + \varepsilon - \hat{f} + \mathrm{E}[\hat{f}] - \mathrm{E}[\hat{f}])^2\right] \\
&= \mathrm{E}\left[(f - \mathrm{E}[\hat{f}])^2\right] + \mathrm{E}[\varepsilon^2] + \mathrm{E}\left[(\mathrm{E}[\hat{f}] - \hat{f})^2\right] + 2\,\mathrm{E}\left[(f - \mathrm{E}[\hat{f}])\varepsilon\right] + 2\,\mathrm{E}\left[\varepsilon(\mathrm{E}[\hat{f}] - \hat{f})\right] + 2\,\mathrm{E}\left[(\mathrm{E}[\hat{f}] - \hat{f})(f - \mathrm{E}[\hat{f}])\right] \\
&= (f - \mathrm{E}[\hat{f}])^2 + \mathrm{E}[\varepsilon^2] + \mathrm{E}\left[(\mathrm{E}[\hat{f}] - \hat{f})^2\right] + 2(f - \mathrm{E}[\hat{f}])\,\mathrm{E}[\varepsilon] + 2\,\mathrm{E}[\varepsilon]\,\mathrm{E}\left[\mathrm{E}[\hat{f}] - \hat{f}\right] + 2\,\mathrm{E}\left[\mathrm{E}[\hat{f}] - \hat{f}\right](f - \mathrm{E}[\hat{f}]) \\
&= (f - \mathrm{E}[\hat{f}])^2 + \mathrm{E}[\varepsilon^2] + \mathrm{E}\left[(\mathrm{E}[\hat{f}] - \hat{f})^2\right] \\
&= (f - \mathrm{E}[\hat{f}])^2 + \mathrm{Var}[\varepsilon] + \mathrm{Var}\left[\hat{f}\right] \\
&= \mathrm{Bias}[\hat{f}]^2 + \mathrm{Var}[\varepsilon] + \mathrm{Var}\left[\hat{f}\right]
\end{aligned}$$

Si noti che la varianza dell'errore irriducibile costituisce il limite inferiore per l'errore di generalizzazione (MSE).

Compromesso bias-varianza: complessità

$$= \mathrm{Bias}[\hat{f}]^2 + \mathrm{Var}[\varepsilon] + \mathrm{Var}\left[\hat{f}\right]$$

Il primo termine è il bias al quadrato, che caratterizza la differenza tra le medie della stima e i valori reali. Un modello con un alto bias non sta catturando il comportamento base della "vera" forma funzionale. Si può immaginare una situazione dove si usa una regressione lineare per modellare una curva sinusoidale. Non importa quanto bene il modello si "adatti" ai dati, non catturerà mai la non linearità inerente a una curva sinusoidale (underfitting).

Il termine medio è la varianza della stima. Determina la deviazione della stima del modello medio quando vengono provati diversi dati di addestramento. In particolare, un modello con un'elevata varianza suggerisce che sia troppo adattato ai dati di allenamento (overfitting).

Il termine finale è noto come errore irriducibile . È il limite inferiore minimo per il test MSE. Dato che abbiamo accesso solo ai punti dati di addestramento (inclusa la casualità associata ai valori ϵ) non è mai possibile ottenere un adattamento "più accurato" di quello che offre la varianza dei residui.

Man mano che la flessibilità aumenta, il bias tenderà a diminuire rapidamente (più velocemente di quanto la varianza possa aumentare) e quindi vediamo un calo nel test MSE. Tuttavia, man mano che la flessibilità aumenta ulteriormente, vi è meno riduzione del bias (perché la flessibilità del modello può adattarsi facilmente ai dati di addestramento) mentre la varianza aumenta rapidamente, a causa del sovradimensionamento del modello.

L'obiettivo finale del machine learning è cercare di ridurre al minimo il test MSE previsto, ovvero scegliere un modello di machine learning statistico che abbia contemporaneamente una bassa varianza e un basso bias.